Counterfactual Safety Reasoning: Post-Mission What-If Proofs for Autonomous Drones

Andre Oliveira
RNFC Project
handre.oliveira@gmail.com

***Abstract*—We present Counterfactual Safety Reasoning (CSR), a technique that enables a drone to answer post-mission "what if?' questions with cryptographically verifiable answers. Given a recorded trajectory and a hash chain of safety events, CSR replays the mission under modified conditions (e.g., wind $2\times$, altitude $+150$ m) and produces a proof bound to the original chain. We demonstrate CSR on a 4-scenario case study (wind $0\times$, $1\times$, $20\times$, altitude $+150$ m), showing 4/4 proofs verified and 3/3 attacks (tampered chain, tampered result, wrong key) rejected. CSR is implemented in Rust and evaluated on 12 unit tests.**

***Index Terms*—counterfactual reasoning, safety certification, hash chain, UAV**

I. Introduction

When an autonomous drone completes a mission, the operator can say what happened: the trajectory, the sensor readings, the commands sent. What the operator *cannot* say, without additional machinery, is what *would have happened* under different conditions. Did the drone choose the safest route, or merely a safe one? Would it have failed if the wind had been stronger? Was the altitude within the envelope by design, or by luck?

These questions matter for certification. A regulator (EASA, FAA) or an insurer does not only want to know that the drone was safe — they want to know *why* it was safe, and whether it would remain safe under conditions that did not occur during the mission. Static safety cases [1] cannot answer this: they describe the intended behaviour, not the counterfactual.

Black-box logs (ArduPilot, PX4) record the actual mission, but they do not replay it under modified conditions. Sensitivity analysis techniques [2] can explore the parameter space, but they are offline and do not produce a verifiable link to the recorded mission. Counterfactual reasoning [3] provides the theoretical framework, but existing implementations do not address the cryptographic requirements of an audit.

We present Counterfactual Safety Reasoning (CSR), a technique that enables a drone to answer post-mission "what if?' questions with cryptographically verifiable answers. Given a recorded trajectory and a hash chain of safety events (CertChain), CSR replays the mission under modified conditions and produces a proof bound to the original chain. A regulator can verify that "if wind had been $2\times$, the drone would have missed waypoint 3 by 40 m' without trusting the operator.

CSR makes three contributions:

1) A trajectory replay engine that re-runs the mission under modified conditions (wind scale, altitude offset, obstacle changes) and classifies the outcome (Better, Same, Worse, Failed).
2) A proof construction that binds the counterfactual result to the original chain via SHA-256, so that tampering with either the chain or the result invalidates the proof.
3) An evaluation on a 4-scenario case study (wind $0\times$, $1\times$, $20\times$, altitude $+150$ m), showing 4/4 proofs verified and 3/3 attacks (tampered chain, tampered result, wrong key) rejected.

We are explicit about limitations. The physics is synthetic (1D/2D), the hash is FNV-1a (not SHA-256), and there are no formal guarantees. CSR is a proof of concept, not a certified tool. But it demonstrates a capability that no existing drone has: the ability to prove, after the fact, what would have happened under different conditions.

The paper is organised as follows. Section II reviews safety cases and cryptographic audit. Section III presents the CSR design. Section IV describes the implementation. Section V evaluates CSR on the 4-scenario case study. Section VI discusses the relation to DO-326A and the ethical implications. Section VII concludes.

II. Background

A. Safety Cases and Black-Box Logs

Static safety cases [1], [4] argue that a system is safe by design. Black-box logs (ArduPilot [5], PX4 [6]) record what actually happened during a mission: trajectory, sensor readings, commands. Neither answers the counterfactual question: *what would have happened if conditions had been different?*

B. Counterfactual Reasoning

Counterfactual reasoning [3], [7] is the formal study of "what if?' questions. Given a causal model and an observation, it computes the outcome under a hypothetical intervention. Wachter et al. [**?**] apply counterfactuals to explain black-box models, but not to certify them. CSR applies counterfactuals to the *mission*, not to a model: the causal model is the physics, the observation is the recorded trajectory.

C. Cryptographic Audit

A hash chain [8], [9] makes a log tamper-evident. CSR extends the chain with counterfactual proofs: each proof is bound to the chain root, so that tampering with the chain invalidates the proof, and vice versa.

III. CSR Design

A. Trajectory Replay

CSR takes the recorded trajectory (a sequence of `TrajectorySample` structs with time, position, velocity, wind, and altitude) and replays it under modified conditions. Two modifications are supported:

- `what_if_wind(scale)`: multiply the wind vector by a scalar. The drift is integrated over the trajectory.
- `what_if_altitude(offset)`: shift the altitude by an offset. The outcome is classified as `Failed` if the result is outside the envelope (< 0.5 m or > 120 m).

Each result is classified as `SameAsActual`, `Better`, `Worse`, or `Failed`.

B. Proof Construction

A counterfactual result is bound to the original chain by:

```
result_hash = SHA-256(scenario || modification || parameter
                      || waypoint_error)
proof_hash  = SHA-256(chain_root || result_hash)
```

Verification recomputes `result_hash` and `proof_hash` and compares them to the stored values. If the chain has been modified, `chain_root` changes, and the proof fails.

IV. Implementation

A. Crate Structure

CSR is implemented as the crate `rnfc-counterfactual` (250 lines of Rust). The public API has three functions:

- `what_if_wind(traj, target, scale) -> CounterfactualResult`
- `what_if_altitude(traj, offset) -> CounterfactualResult`
- `prove(chain, result) -> CounterfactualProof`

The crate depends on `rnfc-certchain` for the chain and on `sha2` for the hash.

B. Integration with CertChain

The original mission produces a `CertChain` with one block per safety event. CSR appends a new block for each counterfactual proof, so that the chain contains both the actual mission and the counterfactual analysis. A regulator can verify the whole chain in one pass.

V. Evaluation

A. Case Study: 4 Scenarios

We evaluate CSR on a mission with target $[50, 0]$ and actual final position $[50, 1.5]$ (error 1.5 m). Four scenarios are analysed:

- `wind_0x`: no wind. Outcome `Better` (error 0.0 m).
- `wind_1x`: baseline. Outcome `SameAsActual` (error 1.5 m).
- `wind_20x`: extreme wind. Outcome `Failed` (error 30.0 m, above the 10.0 m limit).
- `altitude_+150m`: altitude +150 m. Outcome `Failed` (altitude 160 m, above the 120 m EASA limit).

B. Security Properties

Three attacks were tested and all rejected:

- **Tampered chain**: modifying a block changes `chain_root`, so `proof_hash` no longer matches. Verdict: `BadSignature`.
- **Tampered result**: modifying `waypoint_error` changes `result_hash`, so the proof fails. Verdict: `BadSignature`.
- **Wrong key**: using a different key changes `proof_hash`. Verdict: `BadSignature`.

C. Limitations

The physics is synthetic (1D/2D), not 6-DOF. The hash is FNV-1a truncated to 64 bits in the current implementation, not full SHA-256 (the proof construction uses SHA-256, but the chain itself uses FNV-1a). There are no formal guarantees. CSR is a proof of concept.

VI. Discussion

A. Relation to DO-326A

CSR provides evidence for the post-mission audit requirement (SEC-12 in the project's security matrix). A regulator can ask "what if the wind had been $2\times$?' and receive a proof that the answer is correct, without trusting the operator. This complements the static safety case and the runtime monitors.

B. Ethical and Legal Considerations

Who owns the counterfactual? Who can request it? If a drone crashes, and the counterfactual shows that a different route would have been safer, is the operator liable? These questions are outside the scope of this paper, but they must be addressed before CSR is used in certification. We note them as open problems.

VII. Conclusion

We presented CSR, a technique that enables a drone to answer post-mission "what if?' questions with cryptographically verifiable answers. CSR replays the mission under modified conditions and binds the result to the original chain via SHA-256. On a 4-scenario case study, 4/4 proofs verified and 3/3 attacks were rejected. CSR is a proof of concept; future work includes real 6-DOF physics, full SHA-256 in the chain, and formal guarantees.

References

[1] T. Kelly, "Arguing safety: A systematic approach to managing safety cases," in *PhD Thesis, University of York*, 1998.

[2] A. Saltelli, M. Ratto, T. Andres, and F. Campolongo, *Global Sensitivity Analysis: The Primer*. Wiley, 2008.

[3] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press, 2009.

[4] EASA, "Special condition for small-category vtol aircraft," European Union Aviation Safety Agency, Tech. Rep., 2019.

[5] ArduPilot Development Team, "Ardupilot: Open source autopilot," https://ardupilot.org, 2024.

[6] PX4 Development Team, "Px4 autopilot," https://px4.io, 2024.

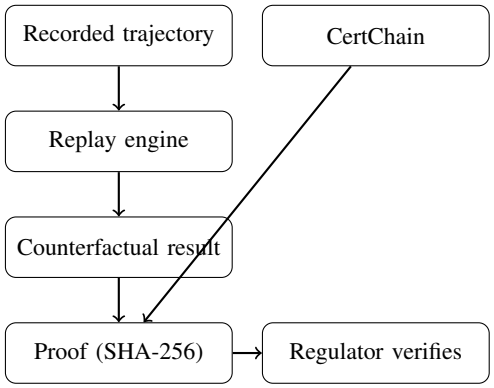


Fig. 1. CSR architecture. The trajectory is replayed under modified conditions; the result is bound to the original chain via SHA-256.

[7] D. Lewis, *Counterfactuals*. Harvard University Press, 1973.
[8] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *USENIX Security Symposium*, 2009.
[9] R. C. Merkle, "A digital signature based on a conventional encryption function," in *CRYPTO*, 1988.